WALCHAND COLLEGE OF ENGINEERING, SANGLI

(Government Aided Autonomous Institute)

WALCHAND LINUX USERS' GROUP

WLUG

Community | Knowledge | Share

# METAMORPHOSIS

LIMITED SEATS

SAILING THROUGH THE CONTAINERS ERA

EXCITING PRIZES

11 & 12 APRIL

- DOCKER DRIFT
- DOCKER DEEPDIVE

- GO KICKSTART
- GO UNLEASHED

REGISTER HERE

SPONSORED BY

FOSS UNITED

CONNECT WITH US

₹349 | ₹649 | ₹899

MAIN & MINI CCF

FOR ANY QUERY
9579047160
7219685991

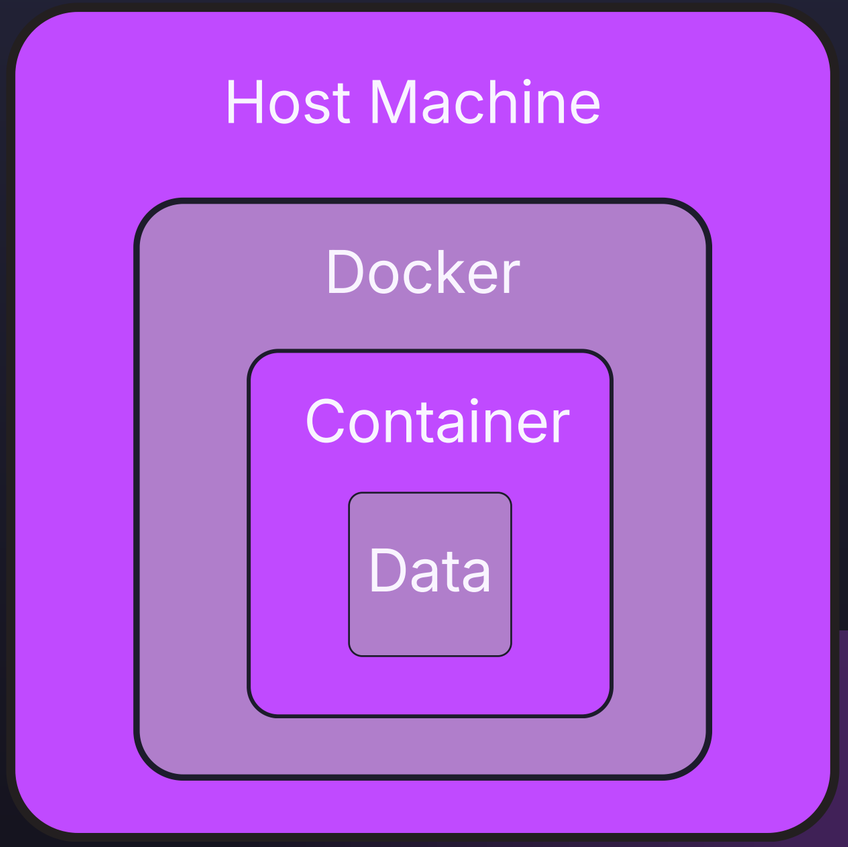
| | | | | | |
|---|---|---|---|---|---|
| Ms. Pooja Bhosale<br>President<br>Walchand Linux Users' Group | Dr. A. J. Umbarkar<br>Staff Advisor<br>Walchand Linux Users' Group | Dr. R. R. Rathod<br>HoD Information Technology | Dr. A. R. Surve<br>Staff Advisor and HoD Computer Science and Engineering | Dr. M. M. Khot<br>Dean (SW) | Dr. S. S. Rathod<br>Director |

WLUG
COMMUNITY | KNOWLEDGE | SHARE

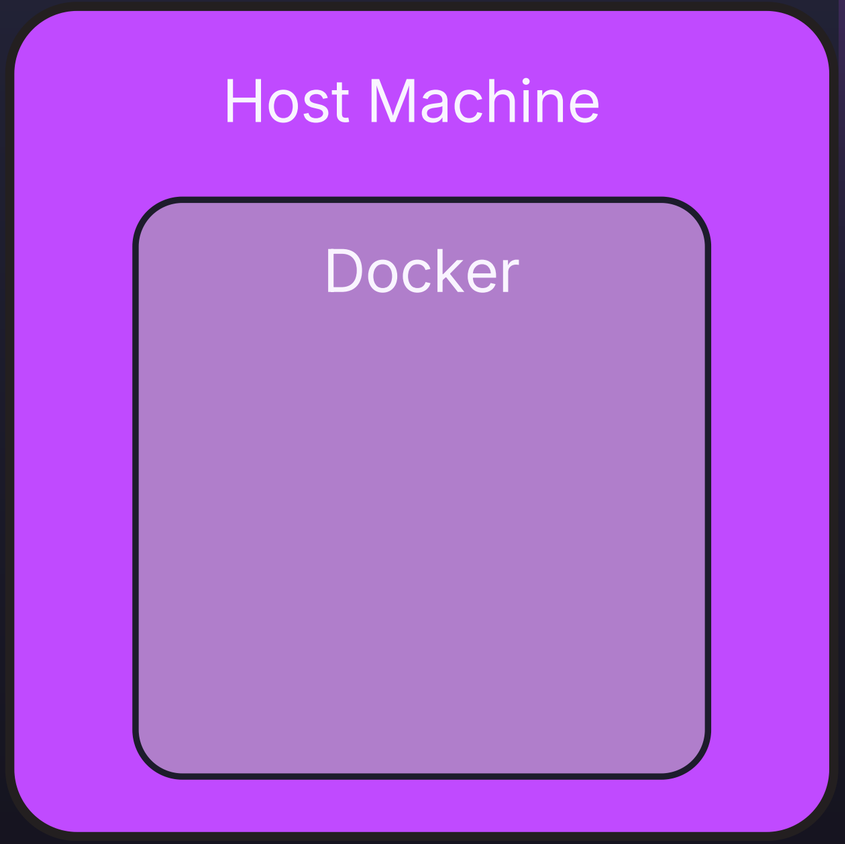
# DOCKER DATA MANAGEMENT

# DOCKER DATA STORAGE

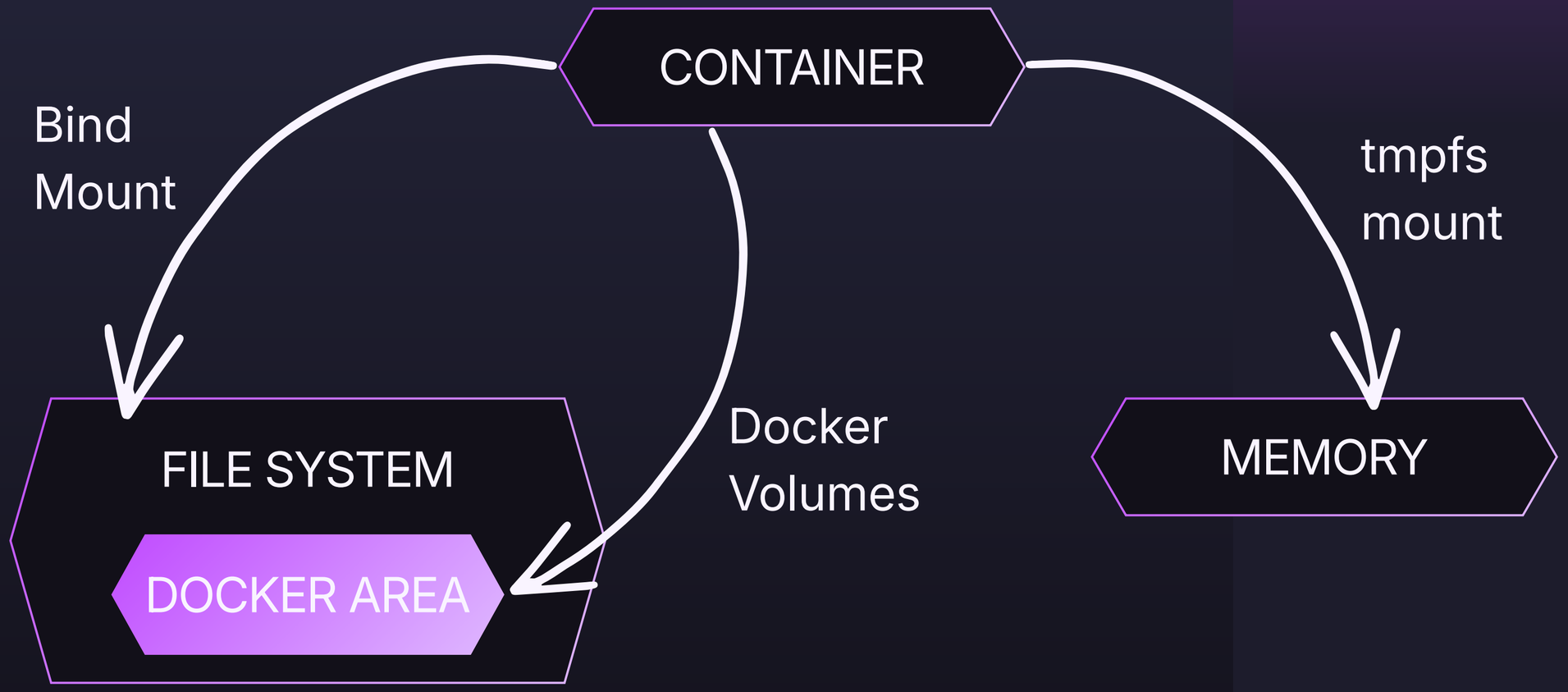
- Each container has a writable layer
- All new files and changes are stored in this layer
- After deleting the container, all data is lost



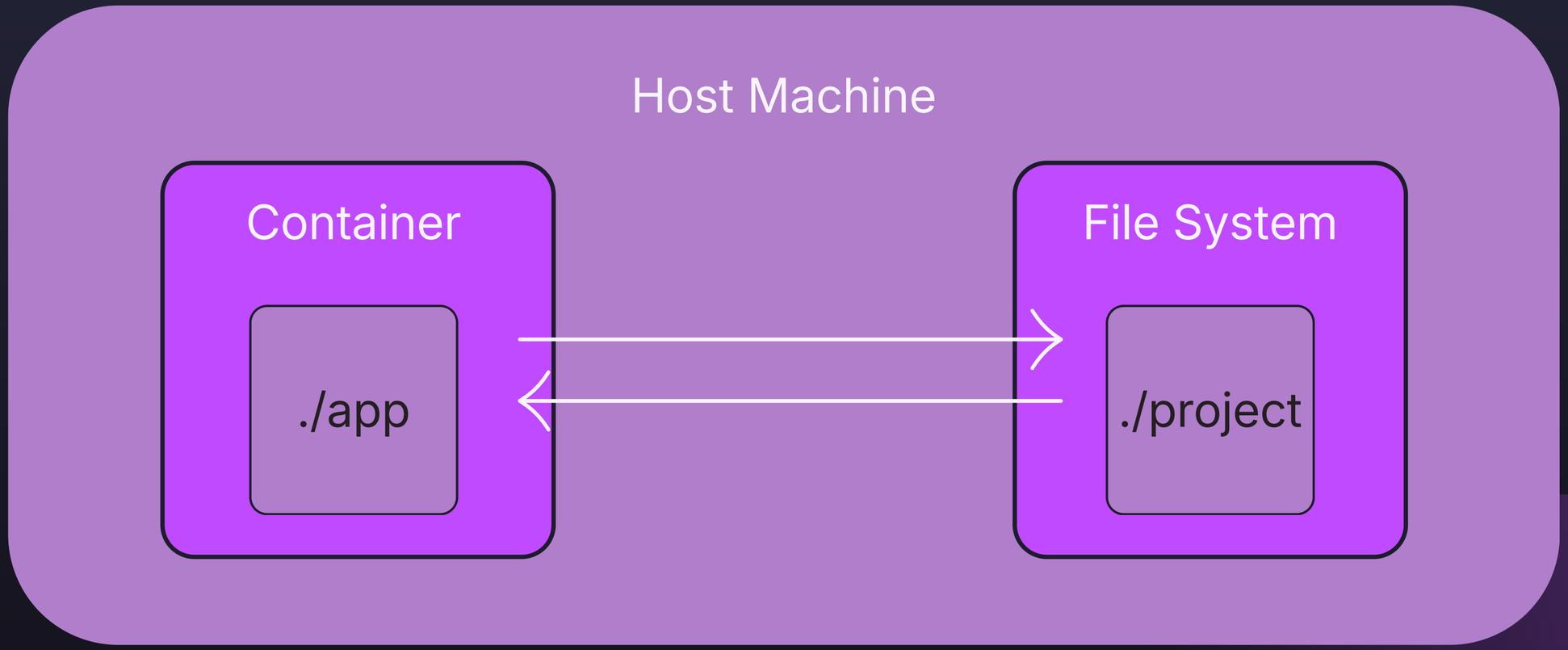
Container is deleted!



# SOLUTIONS


CONTAINER
Bind
Mount
Docker
Volumes
tmpfs
mount
FILE SYSTEM
DOCKER AREA
MEMORY

# BIND MOUNTS



# BIND MOUNT

- Linking a container folder to your local machine
- Bind mounts are stored anywhere on the host system
- Rely on the host file system
- Not managed by docker

# BIND MOUNT

- Syntax

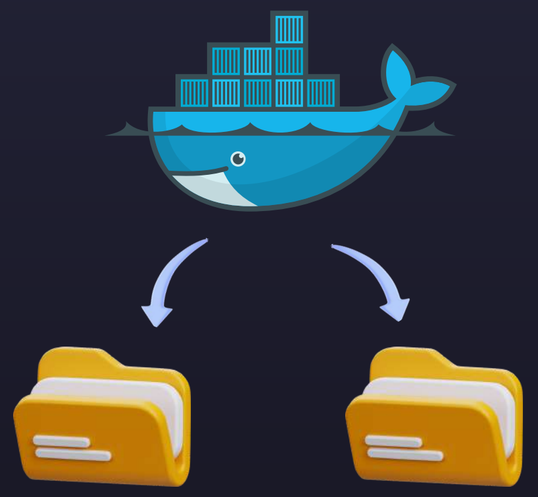
```
docker run -v <host-path>:<container-path> <image-name>
```

OR

```
docker run --mount type=bind,source=$(pwd),target=/app ubuntu
```

# DOCKER VOLUMES

- Volumes are persistant data storage for containers
- Docker manages the volumes
- Data resides on the host file system, typically under var/lib/docker/volumes/vol-name/_data



# DOCKER VOLUMES

- Create volume

```
docker volume create <volume-name>
```

- List volumes

```
docker volume ls
```

- Inspect volume

```
docker volume inspect <volume-name>
```

# BACKGROUND

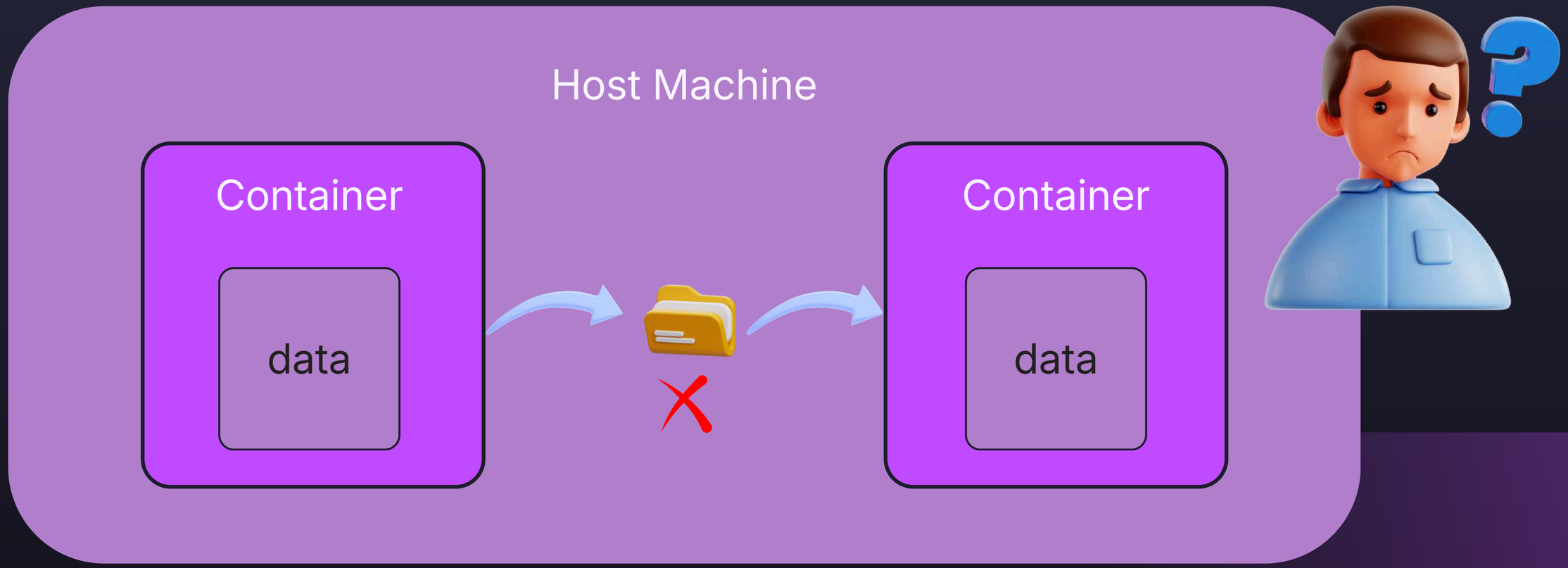
- Remove volume

```
docker volume rm <volume-name>
```

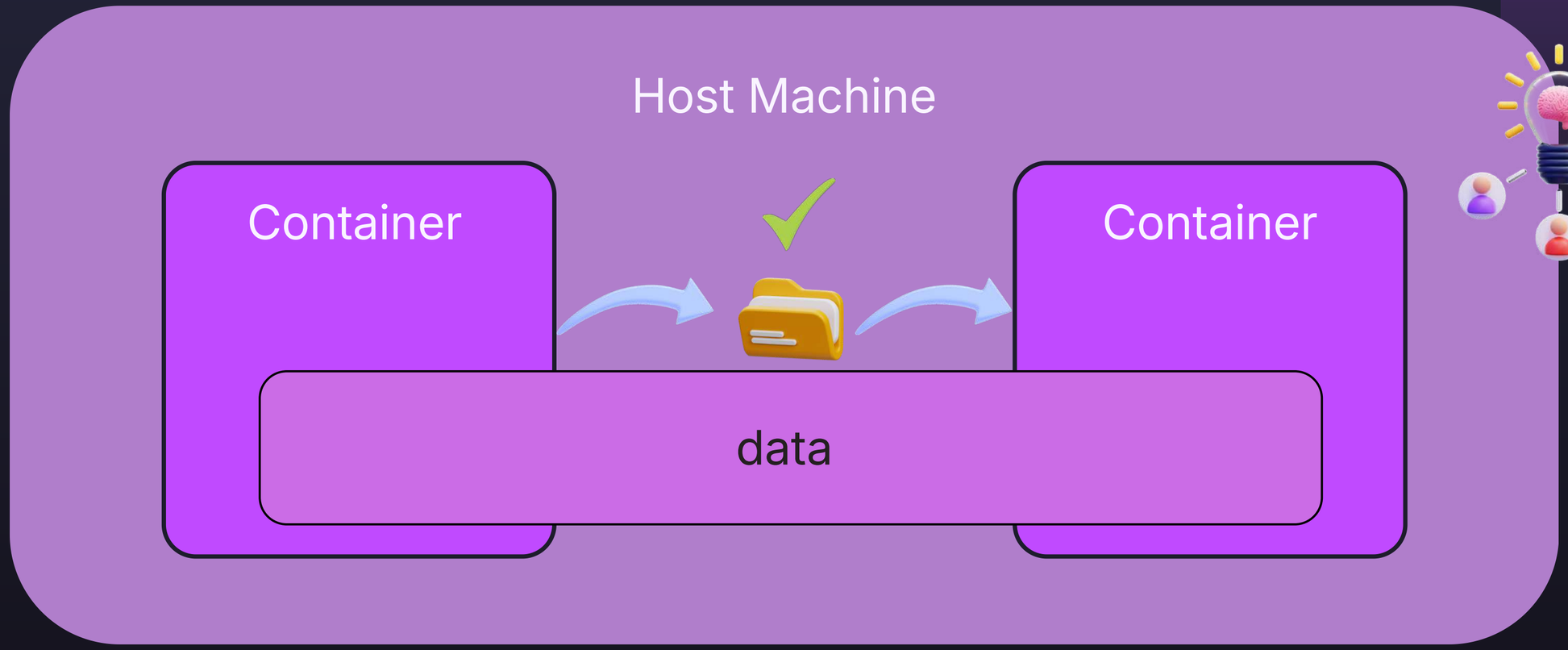
- Prune

```
docker volume prune
```

# SHARED VOLUMES



# SHARED VOLUMES



# SHARED VOLUMES

- Centralized data sharing between containers
- Ensures data consistency accross multiple containers
- Changes made by one container are instantly available to others

WLUG
COMMUNITY | KNOWLEDGE | SHARE

# DOCKER FILE

```
# Base Image
FROM <base_image>:<tag>

# Set working directory
WORKDIR /app

# Copy files from host to container
COPY . /app

# Install dependencies
RUN <command_to_install_dependencies>

# Default command
CMD ["executable", "arg1", "arg2"]
```

# FROM

- Defines the base image for the Docker container
- Every Dockerfile must start with FROM
- Acts as the foundation for your environment

# WORKDIR

- Sets the working directory inside the container
- All following commands run from this directory
- Creates the directory if it doesn’t exist

# COPY

- Copies files from host machine → container
- . means current directory
- Used to bring application code into container

# RUN

- Executes commands during image build
- Used to install dependencies
- Creates a new layer in the image

# CMD

- Defines the default command when container runs
- Only one CMD is allowed
- Can be overridden at runtime

WLUG
COMMUNITY | KNOWLEDGE | SHARE

# DOCKER NETWORKS

# DOCKER NETWORKS

- A Docker network is a _____ network that allows Docker containers to communicate with each other and with the outside world.

# DOCKER NETWORKS

- A Docker network is a virtual network that allows Docker containers to communicate with each other and with the outside world.

# WHY TO LEARN THEM?

- One rarely runs the containers alone
- Containers need to communicate with each other

# MONOLITHIC ARCHITECTURE

- Entire application built as one single large system where all components are tightly connected
- Everything runs together in one codebase and usually one deployment unit, i.e. container

# WHY NOT TO STICK TO IT?

- Hard to scale
- Slow deployments
- Tight coupling
- Difficult maintenance of code
- SPOF - single point of failure

# WHY MICROSERVICES?

- Independent scaling
- Independent deployment
- Better fault isolation
- Faster development
- No SPOF

# PHILOSOPHY

- One should stick to micro-services architecture:
  - Breaking applications into small independent services
  - Each service runs in its own container
- Example: An e-commerce website with authentication, payment, order and database service seperately running

Government of India currently operates using 54 different ministries, rather than operating as a single giant body.

For example:

- Agriculture and Farmers' Welfare
- Home
- Finance
- External Affairs

WLUG
COMMUNITY | KNOWLEDGE | SHARE

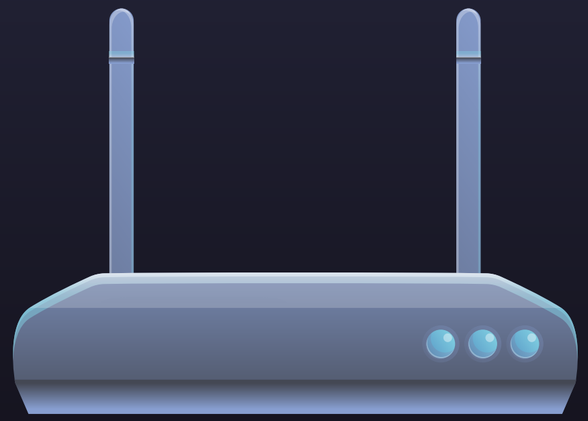
# BASIC TERMINOLOGIES

# IP ADDRESS

- Internet Protocol address
- A unique numerical address assigned to each container in a network
- e.g. 172.17.0.2

# PORT

- A communication endpoint used on by a service on a container
- e.g. 172.17.0.2:8080

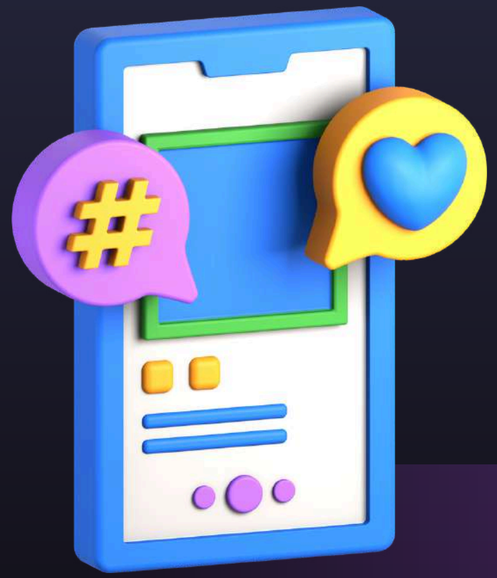


# PORT FORWARDING

- A port can forward requests to another port
- e.g. 192.168.2.3:8080 → 172.17.0.2:80

# NC

- A powerful utility for network debugging, testing, and data transfer
- Called as the **Swiss Army knife of networking**



# NC

- Opens a TCP connection to port 80 (HTTP)

```
nc example.com 80
```

GET / HTTP/1.1
Host: example.com

# MESSAGE TRANSFER (NC)

- Listener `nc -l 1234`
- Sender `nc 127.0.0.1 1234`

# CURL

- A command-line tool used to transfer data to or from a server
- Used for testing APIs, downloading files, and debugging network requests

```
curl example.com
```

WLUG
COMMUNITY | KNOWLEDGE | SHARE

# TYPES OF NETWORKS

# DEFAULT NETWORKS

- Default network on system upon installation
- Uses bridge network driver
- `docker run -it -d ubuntu sh`

# USER DEFINED NETWORKS

- Useful to separate groups of containers that should have full access to each other, but restricted access to containers in other groups

Website A

Website B

WLUG
COMMUNITY | KNOWLEDGE | SHARE

# NETWORK DRIVERS

# BRIDGE

- Creates a virtual private network on a single host
- Containers connected to the same bridge network can communicate with each other
- Isolates network from other networks

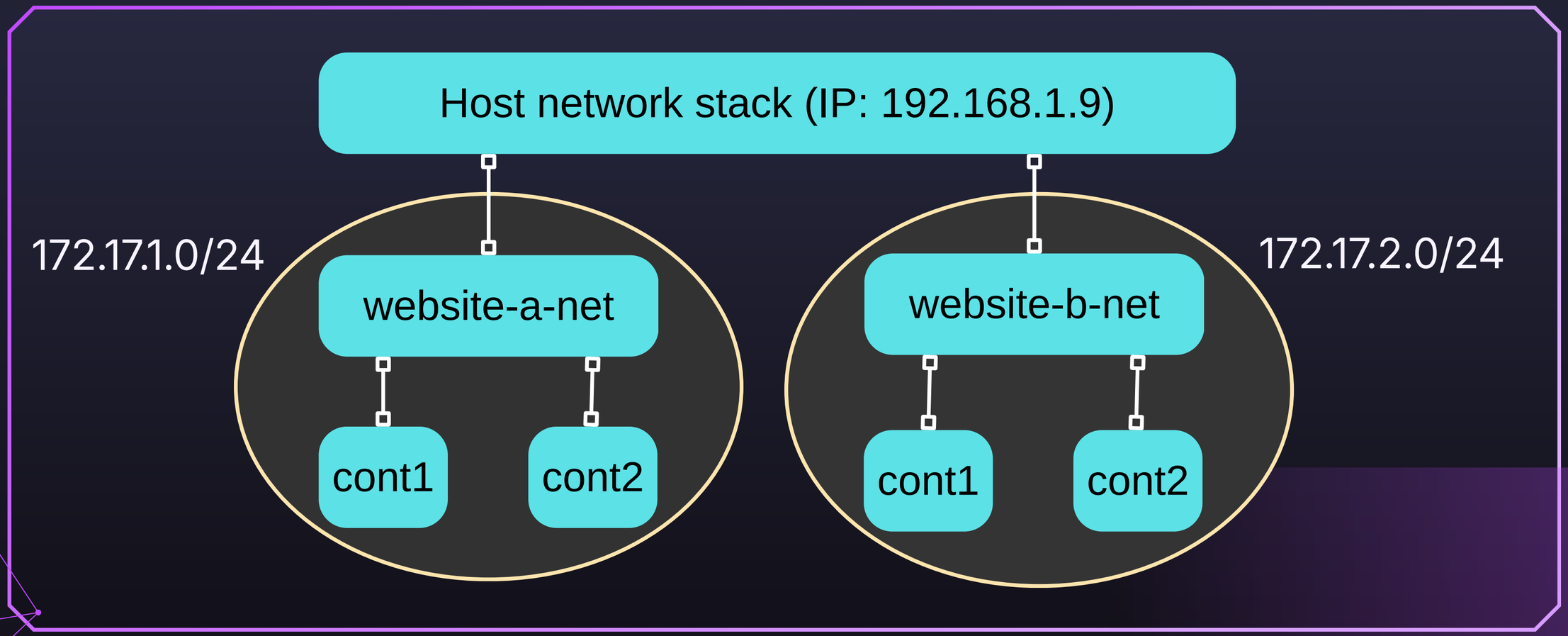
# BRIDGE

- docker network ls
- docker network inspect bridge
- docker network create testing

# BRIDGE

- docker network create --subnet 172.23.0.0/16
- docker network connect my-network container_name
- docker container inspect container_name
- docker network disconnect my-network container_name

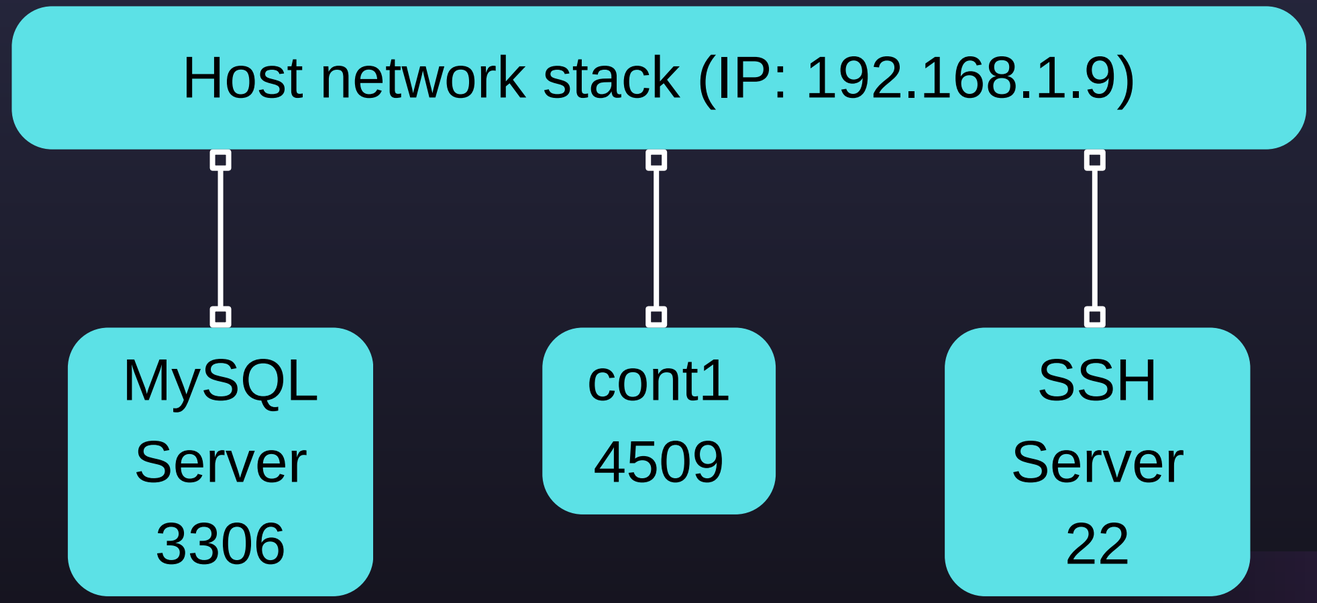
# BRIDGE


Host network stack (IP: 192.168.1.9)
172.17.1.0/24
website-a-net
cont1
cont2
172.17.2.0/24
website-b-net
cont1
cont2

# HOST

- Allows a container to share the host machine's network stack
- Container uses the host's IP address and ports directly instead of having its own network interface
- `docker run --network host -d nginx`

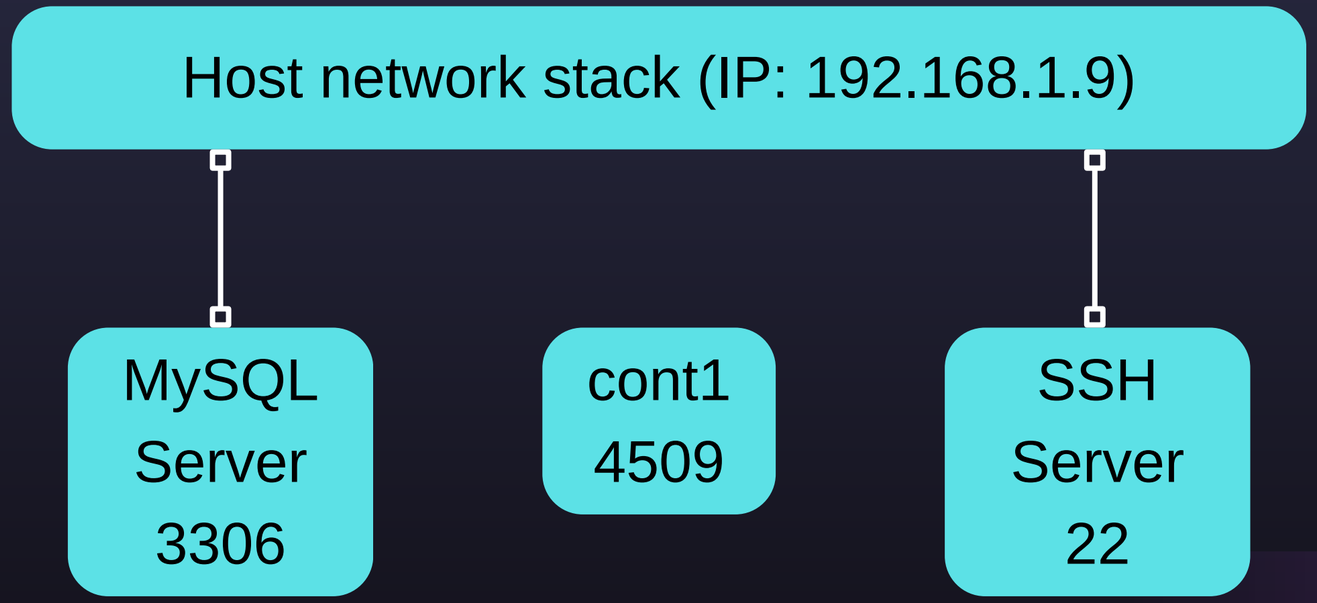
# HOST


Host network stack (IP: 192.168.1.9)
MySQL Server 3306
cont1 4509
SSH Server 22

# NONE

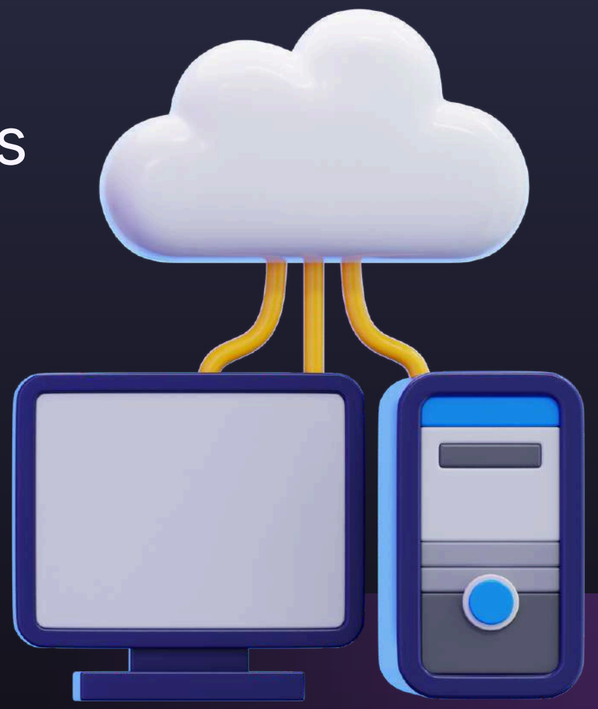
- Disables networking for a container
- Container has no external network access and only a loopback interface (lo)
- `docker run --network none -it -d ubuntu sh`

# HOST


Host network stack (IP: 192.168.1.9)
MySQL Server 3306
cont1 4509
SSH Server 22

# OVERLAY

- Creates a virtual network that connects containers running on different Docker hosts
- Allows them to communicate as if they were on the same network

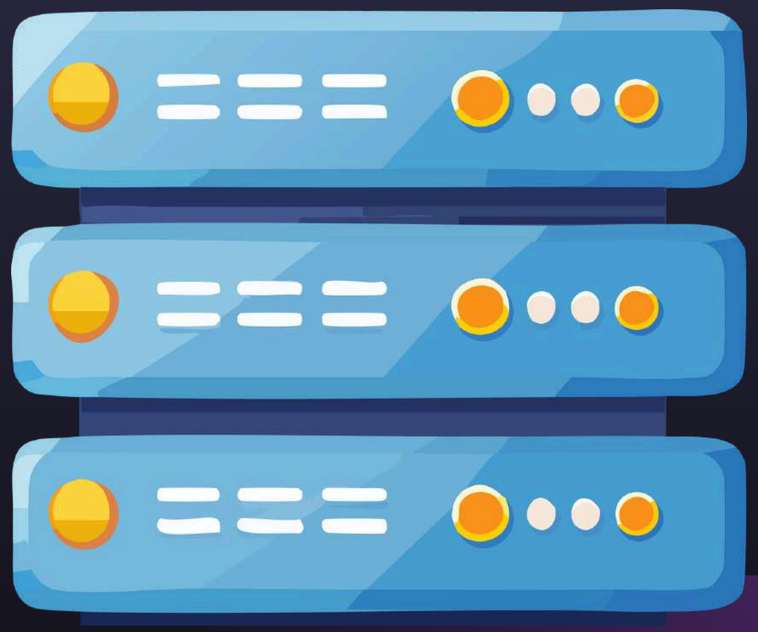


# IPVLAN

- Allows containers to communicate directly with the physical network using IP addresses from the host network
- Share the same MAC address as the host interface

# MACVLAN

- Allows containers to connect directly to the physical network, giving each container its own unique MAC address and IP address
- They appear like separate physical devices on the network



WLUG
COMMUNITY | KNOWLEDGE | SHARE

# MULTI-CONTAINER SETUP

# PYTHON ANALYSIS TOOL

- A Python script that performs a specific data processing task runs within a container with all its dependencies

# PYTHON ANALYSIS TOOL

- It now needs to connect to a database
- Suddenly, you now need to manage a database inside a container itself
- For login service, you need to add that mechanism inside container

# REMEMBER MICROSERVICES?

- Keep the services seperated
- Run a seperate container for analysis tool
- Run a seperate container for database
- Run seperate login service
- Connect them all using a Docker network

# WHAT DID WE ACHIEVE?

- Clear seperation of concerns
- No SPOF
- Better failure management
- Faster development
- Independent scaling

# HOW TO RUN THIS?

- docker run -it -d python bash -v analysis:/app sh
- docker run -it -d mysql bash -v db:/var/lib/mysql
- docker run -it -d login bash -v creds:/var/lib/mysql

WLUG
COMMUNITY | KNOWLEDGE | SHARE

# DOCKER COMPOSE

# WHAT IS IT?

- A tool to run multi-container applications
- Uses a YAML (YAML Ain’t Markup Language) file
- Defines all services in one place
- Runs everything with one command
- Automatically handles networking & dependencies

# BENEFITS OF COMPOSE

- Simplifies multi-container management
- Declarative configuration
- Dependency management
- Easy startup
- Easy stoppage of application

# IMPERATIVE CONFIG

- Specifies each step to achieve a result step-by-step
- Requires detailed commands
- More control over execution
- Relatively difficult to maintain
- Difficult to interpret

# MURPHY'S LAW

*"Anything that can go wrong, will go wrong."*

# DECLARATIVE CONFIG

- Specifies only what the final state should be
- Focus on outcome
- Simpler and more readable
- Automatic maintenance of desired state
- Easier to scale

# HOW TO INSTALL IT?

- sudo apt-get install docker-compose-plugin
- docker compose version

# DEMONSTRATION

- git clone https://github.com/tanishbhongade/wlugmeta2k26-dockercompose

# HOW EVERYTHING WORKS?

WLUG
COMMUNITY | KNOWLEDGE | SHARE

# THANK YOU!

WLUG
COMMUNITY | KNOWLEDGE | SHARE

Scan to join whatsapp community

@WCEWLUG

Scan to join Instagram Account